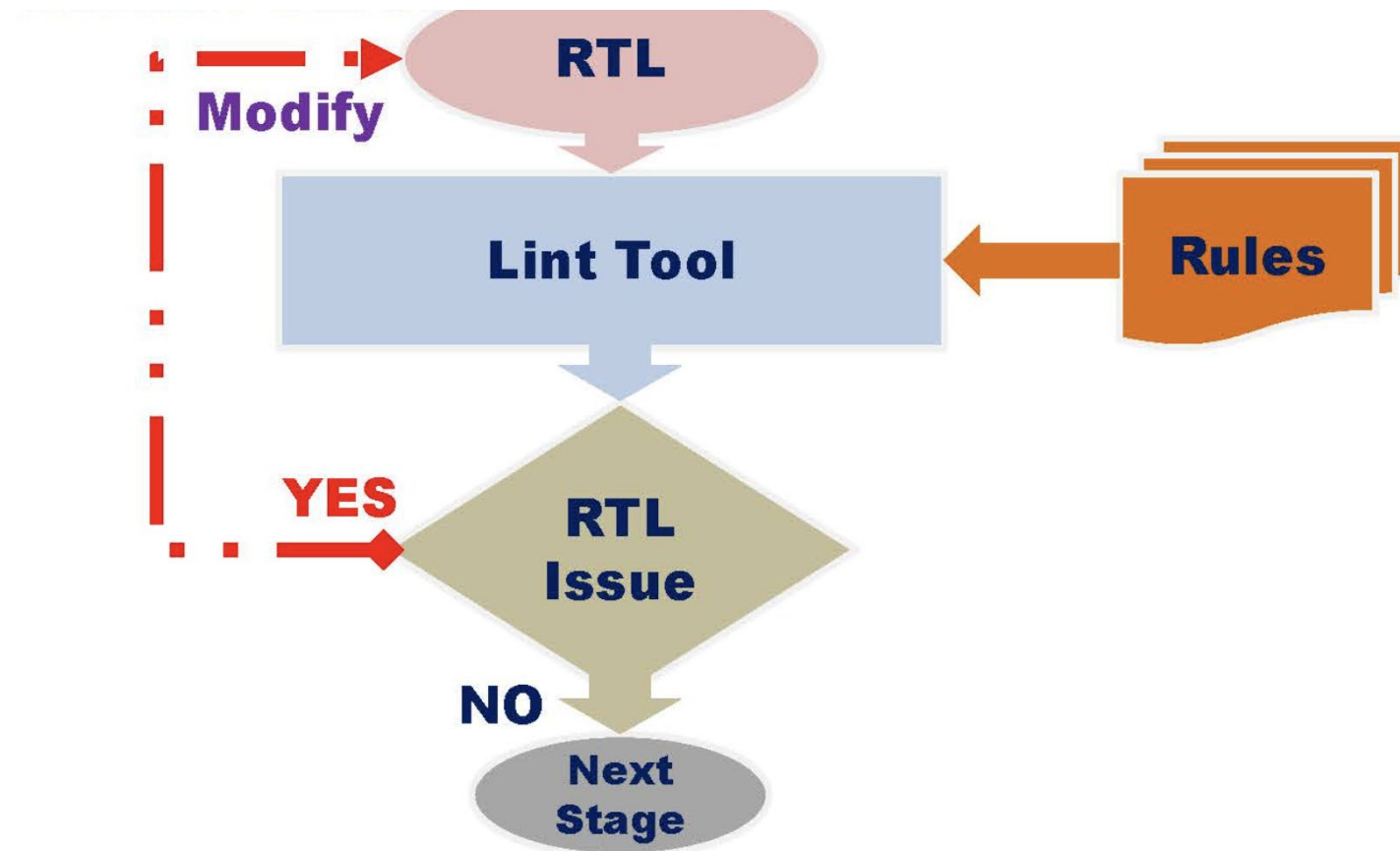


LINT, CDC, UPF training

LINTING

LINTING

Linting is done to check the cleanliness of HDL codes for various EDA tools.



SOME TYPICAL LINT TARGETS

```
module example_2 (out, in);  
output [1:0] out;  
input [2:0] in;  
assign out = in[1:0];  
endmodule
```

Not all elements of the array are read

Undriven nets

```
module example_7(in1, clk, out1);  
input [2:0] in1;  
input clk;  
output [2:0] out1;  
reg [2:0] out1;  
wire [2:0] net;  
always@(posedge clk)  
out1 <= in1 && net;  
endmodule
```

SOME TYPICAL LINT TARGETS (Contd...)

```
module example_9 (clk, reset, d, q);  
input clk, reset, d;  
output q;  
reg q;  
always @(posedge clk or negedge  
        reset)  
begin  
    if (!reset)  
        q = 1'b0;  
    else  
        q = d;  
    end  
endmodule
```

Using Blocking assignments for sequential logic

```
always@(in1 or in2 or in3 or mem)
begin
case(in3)
2'b00 : out[0] = in1 || mem[0];
2'b01 : out[1] = in1 && mem[1];
2'b11 : out[2] = in2 && mem[3];
endcase
end
```

In this example, the case construct does not have all possible cases described and also does not have a default statement.

In this case a latch may be inferred.

It is advisable to examine such cases to avoid creation of unexpected logic.

```
if (reset = '1') then  
  q <= d;  
end if;
```

Latch inference

```
module example_17 ( in1, in2, out1 );  
input in1, in2;  
output out1;  
reg out1;  
always @(in1)  
begin  
out1 = in1&in2;  
end  
endmodule
```

Signal is read inside a combinational always block but it is not included in the sensitivity list.

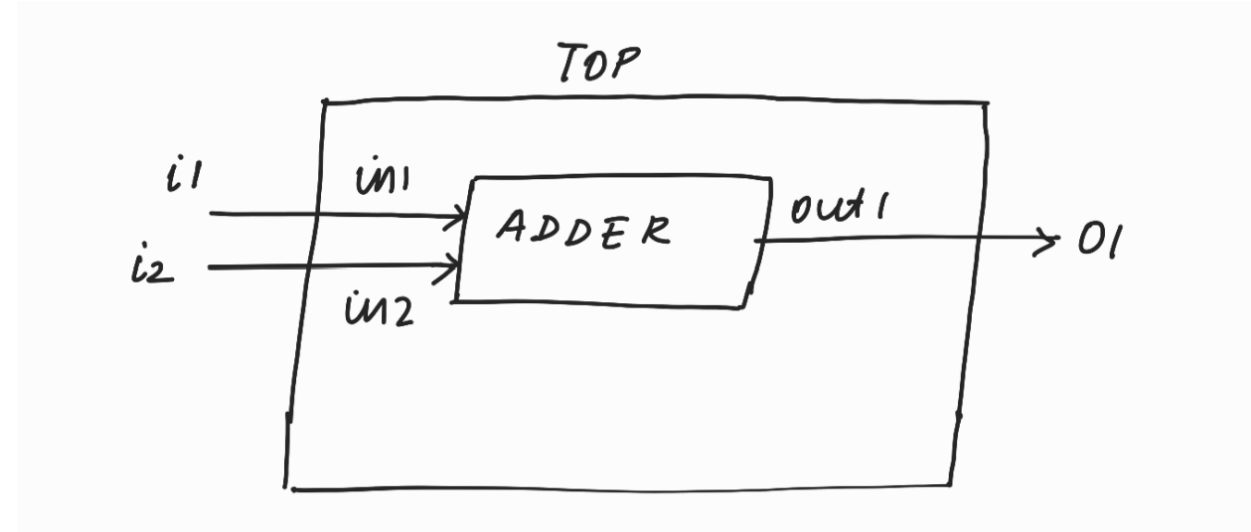

```

module top (in1, in2, o1);
  input [2:0]i1,i2;
  output [3:0] o1;

  adder a1(.in1(i1), .in2(i2), .out1(o1));

endmodule

```



Width Mismatch

```

module adder (in1,in2,out1);
  input [1:0] in1,in2;
  output [2:0] out1;

  assign out1 = in1 + in2;

endmodule

```

```
always @(*) begin  
  multi_dr <= a & b;  
end
```

```
always @(*) begin  
  multi_dr = c & d;  
end
```

Multi-Driver Error

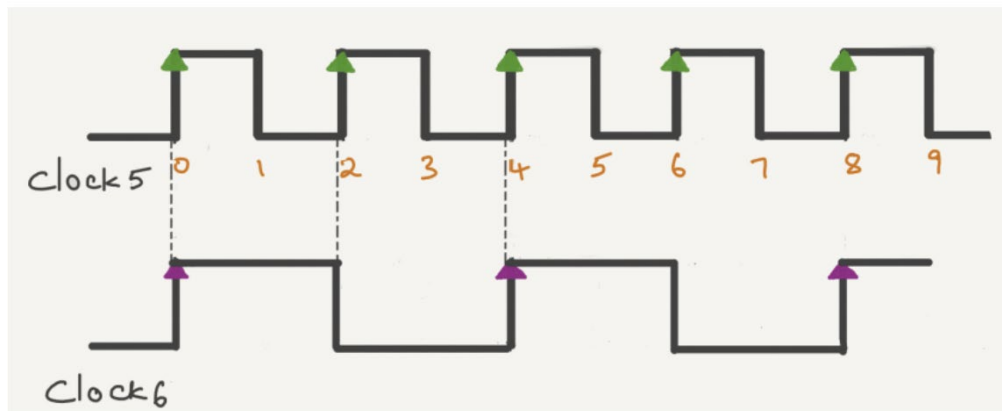
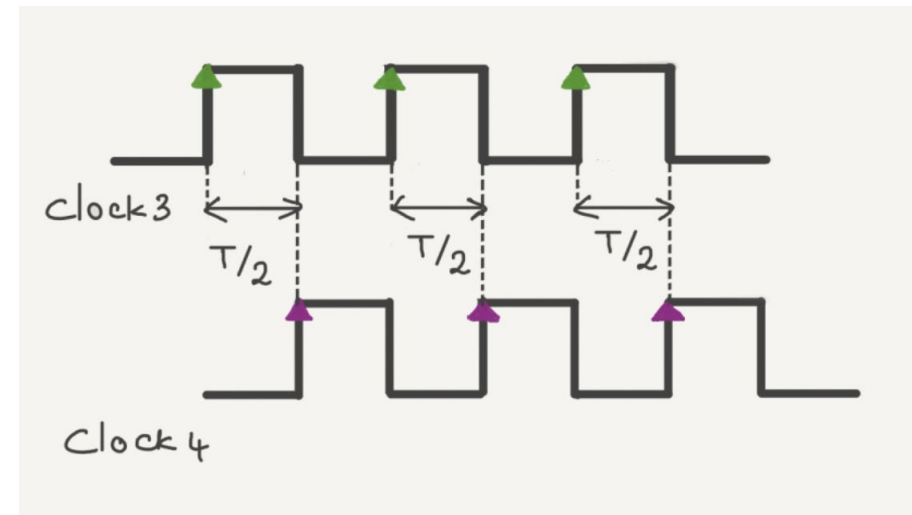
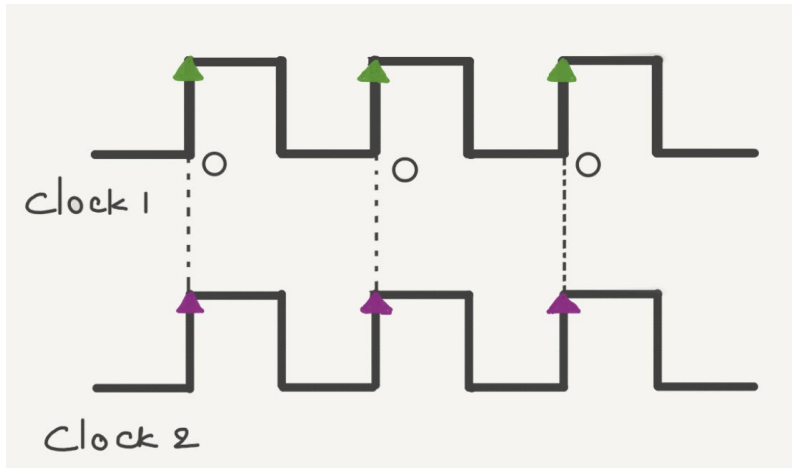
Same signal is driven by 2 always blocks.

Clock Domain Crossing

SYNCHRONOUS CLOCKS

Two clocks are synchronous when both have a same source and have a constant phase relationship between them.

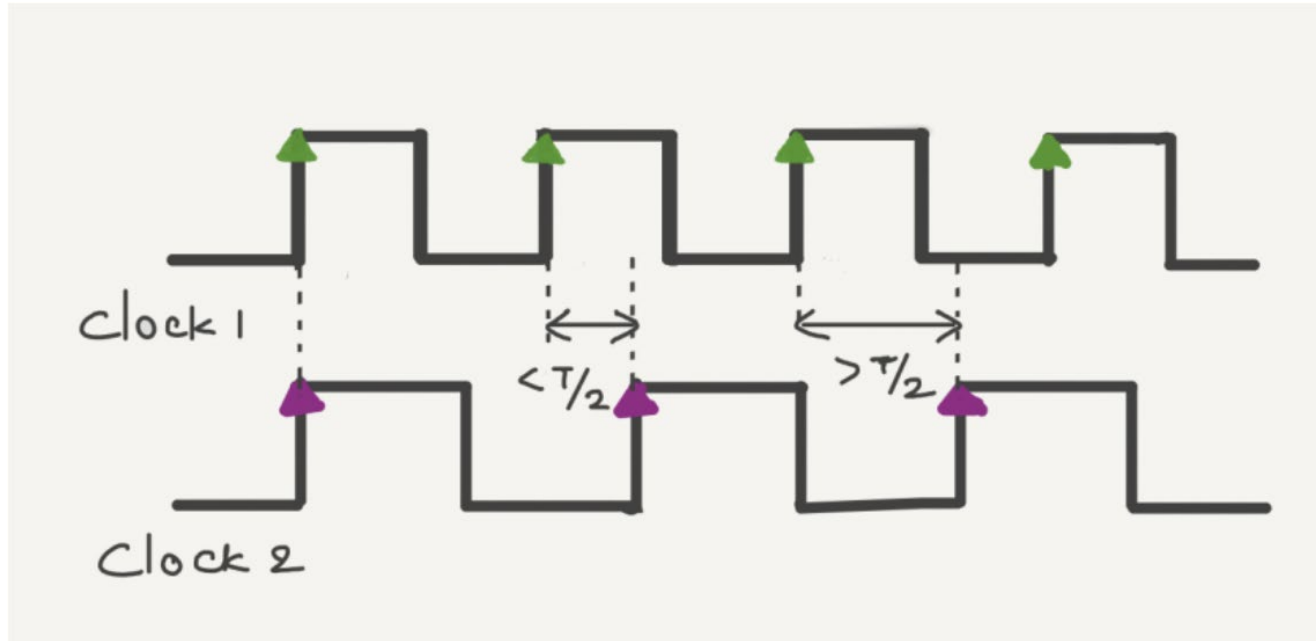
EXAMPLES



ASYNCHRONOUS CLOCKS

Two clocks are asynchronous when they have a different source and variable phase relationship between each other.

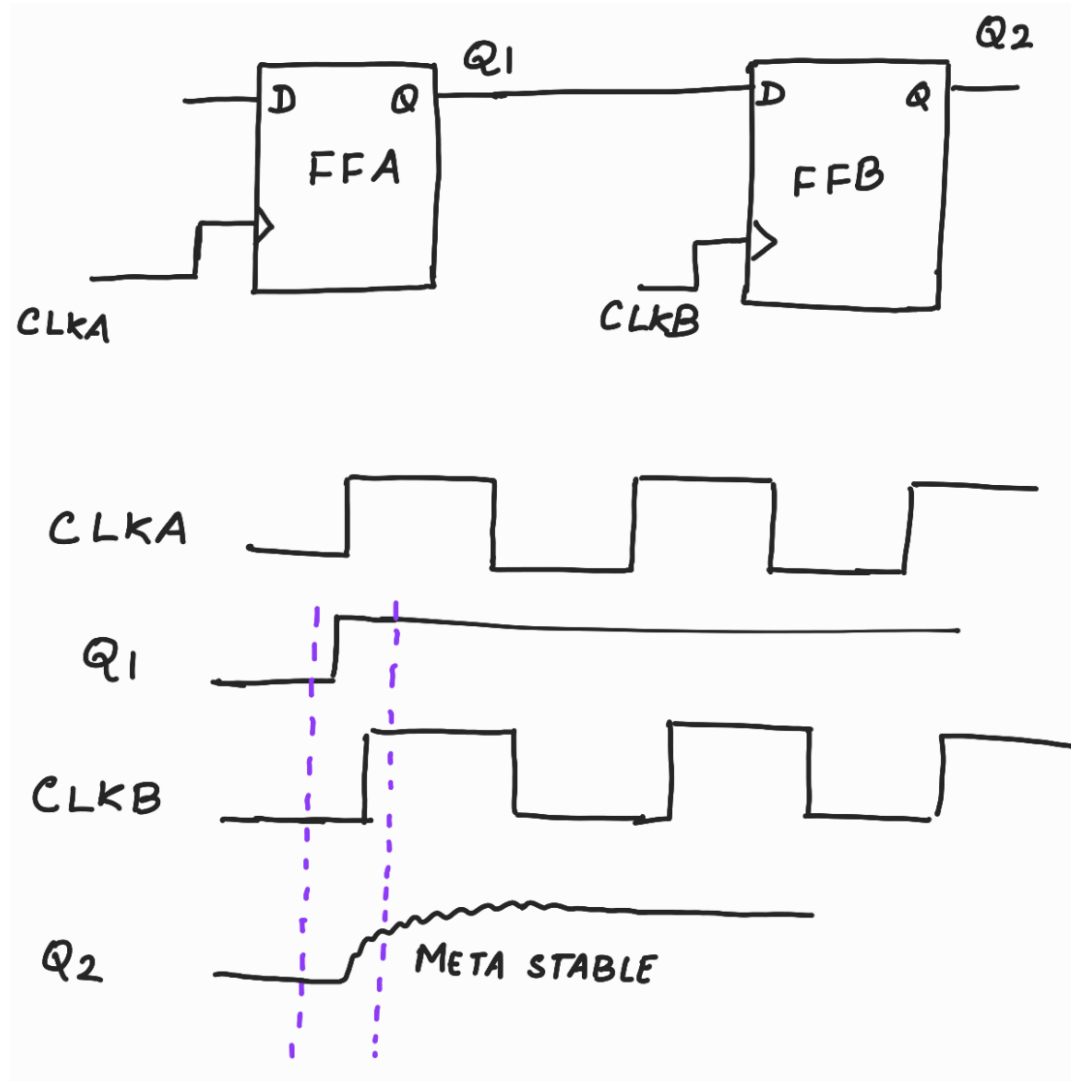
EXAMPLE



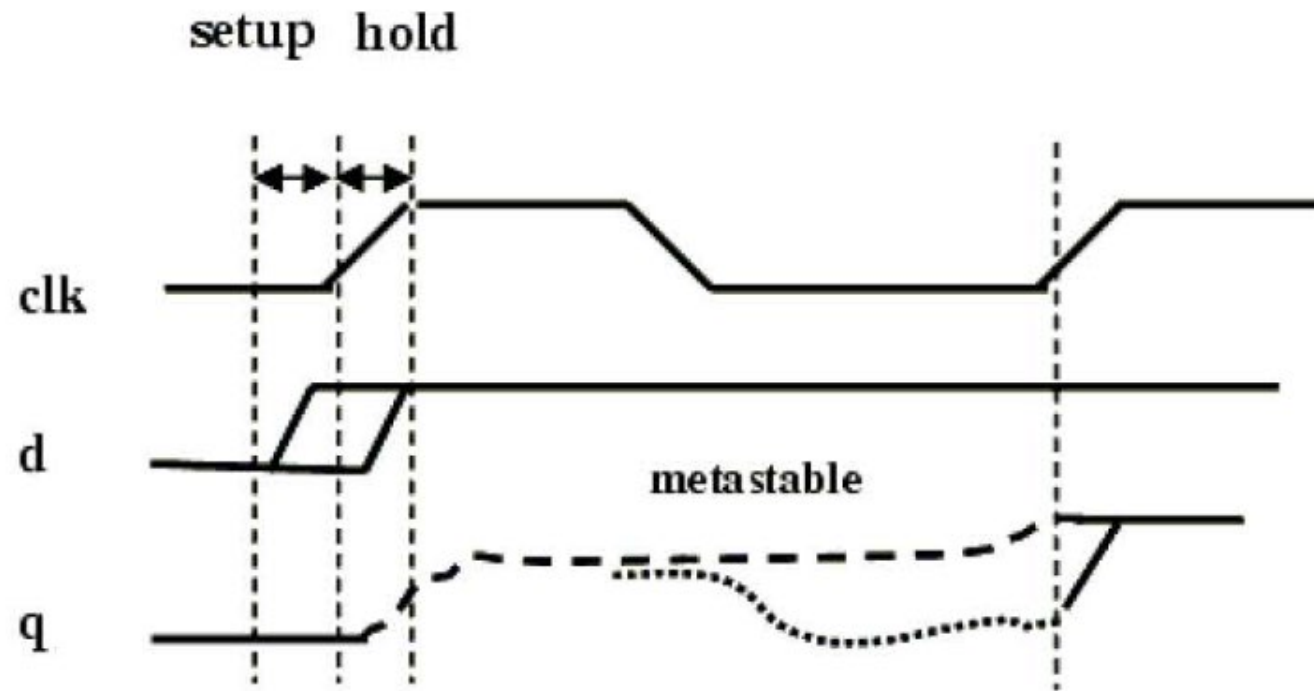
CLOCK DOMAIN CROSSING (CDC)

When logic from two separate clock domains interacts, data corruption can occur if the clock domains are asynchronous. **Clock Domain Crossing (CDC)** is a result of this corruption. CDC is seen only between two asynchronous clocks and not between synchronous clocks.

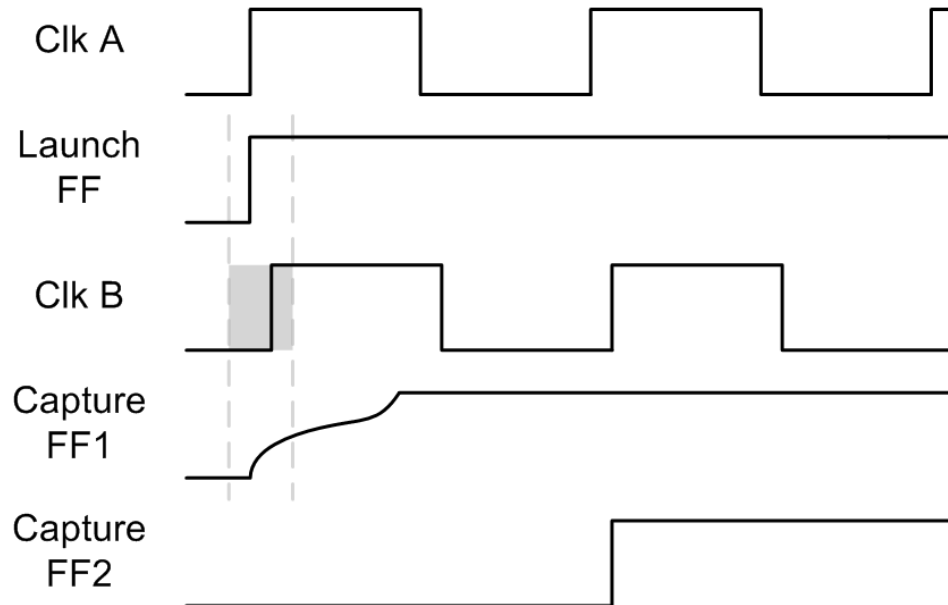
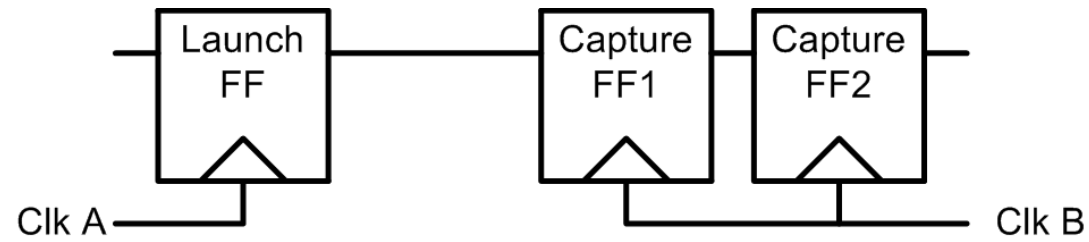
CDC PROBLEM



METASTABILITY



SOLUTION FOR CDC (SINGLE BIT CROSSING)



2 flop Synchronizer

SOLUTION FOR MULTI-BIT CROSSING (USED IN ASYNCHRONOUS FIFO DESIGN)

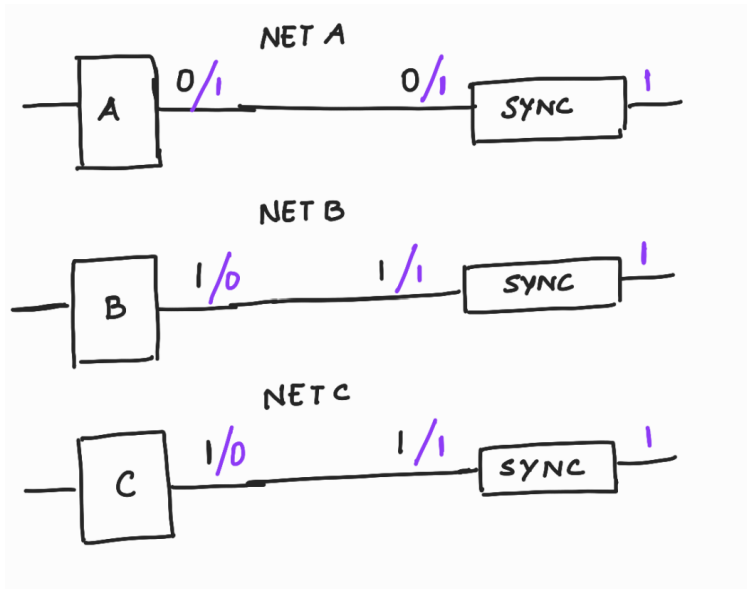
BINARY COUNTER OUTPUT

- In Binary counter multiple bit changes at the same time.

Example : 011 -> 100

- Different synchronizers for binary bits may result in random value.

ILLUSTRATION

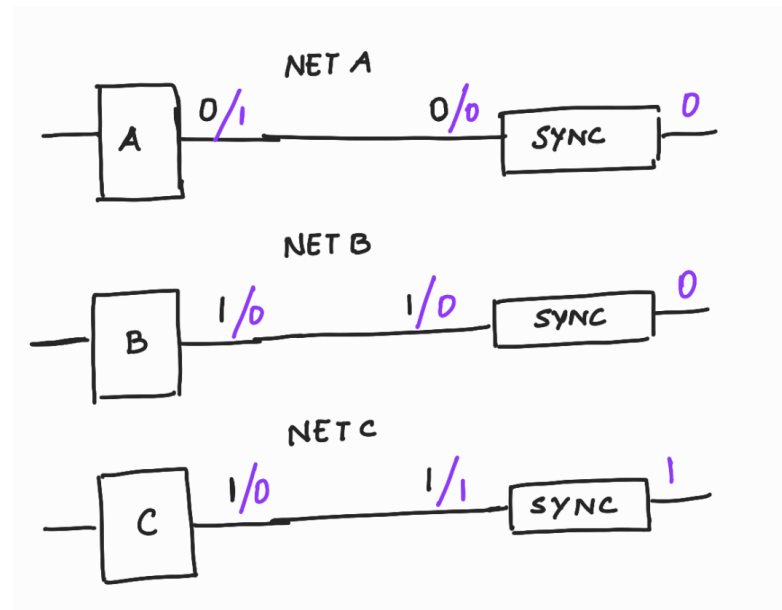


Net A fastest. Net B and Net C slow.

Output is 111

Net B fastest. Net A and Net C slow.

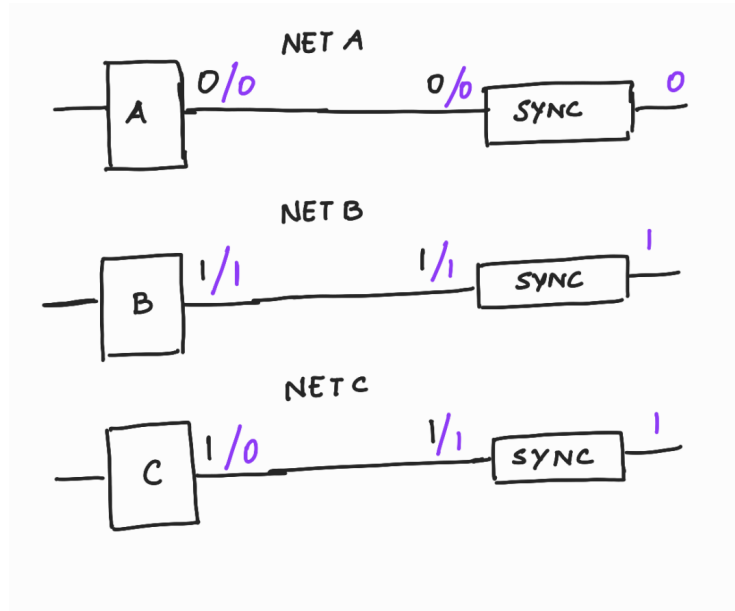
Output is 001



SOLUTION

- Convert binary to gray at the transmit domain.
- Use 2 flop synchronizer at the destination domain to sample the gray counter output from transmit domain.
- In gray counter , only 1 bit changes at a time.
Example : 011 to 010
- The synchronizer output will result in 011 (present value) or next value (010)

ILLUSTRATION

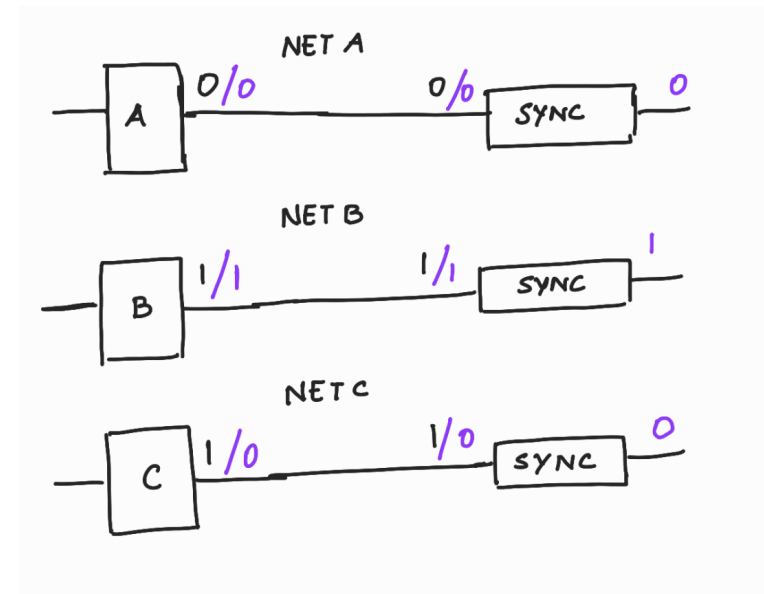


Net A fastest. Net B and Net C slow.

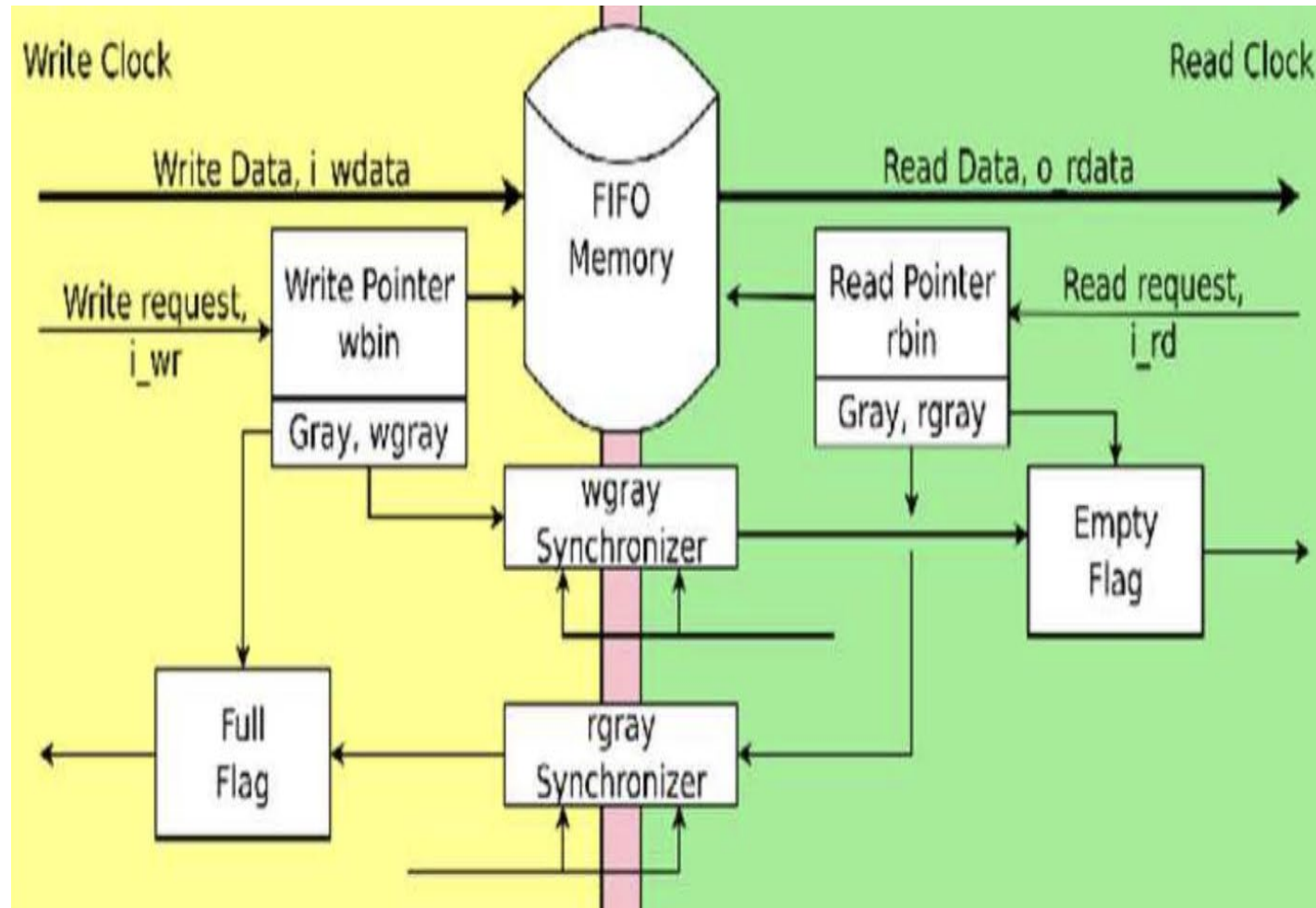
Output is 011

Net A Slow. Net B and Net C fast.

Output is 010



ASYNCHRONOUS FIFO



Low Power Design and UPF

LOW POWER DESIGN

Need of low power design :

In order to obtain longer battery life.

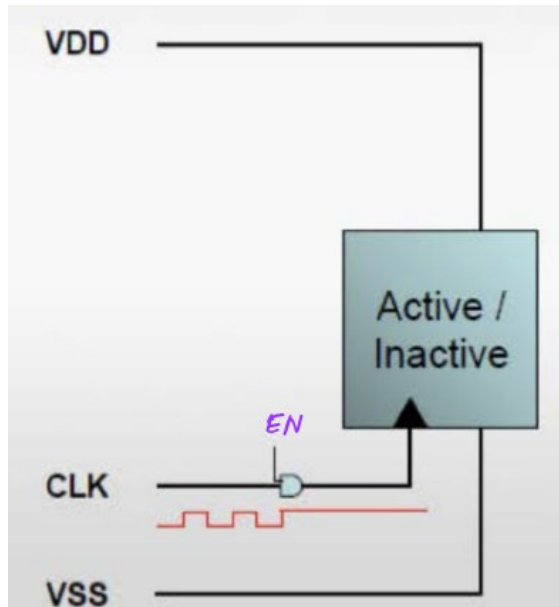
Types of power consumption :

- Dynamic : occurs during switching of transistors
- Static : comprised of leakage current that flows through the transistor when there is no activity.

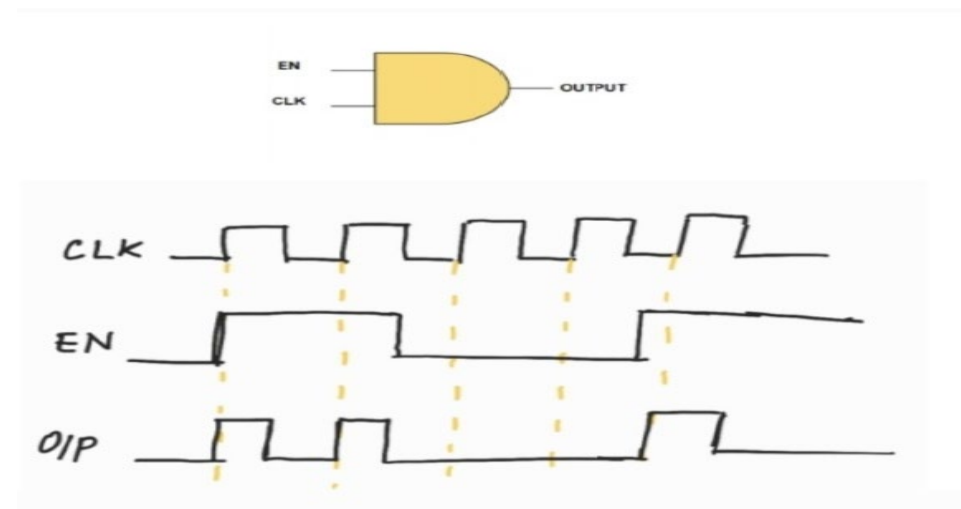
POWER REDUCTION TECHNIQUES

CLOCK GATING :

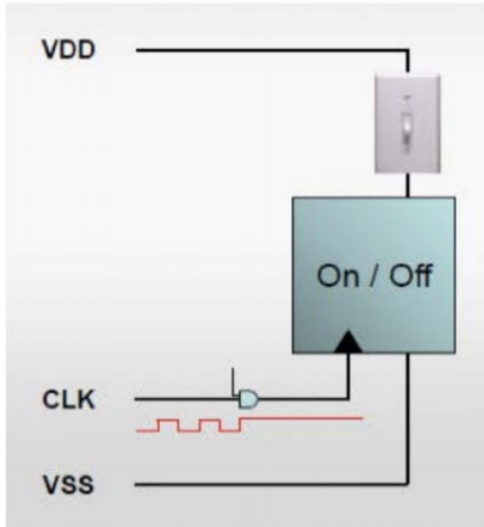
Turning off clock to the module when the module is not in use in order to reduce dynamic power.



When $En = 1$, it allows clock to the modules
When $En = 0$, it stops clock to the modules



POWER REDUCTION TECHNIQUES (Contd...)

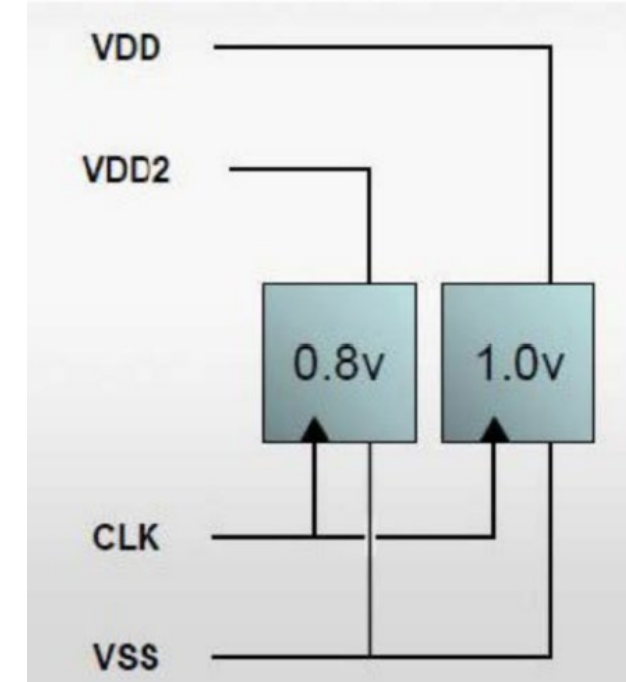


POWER GATING :

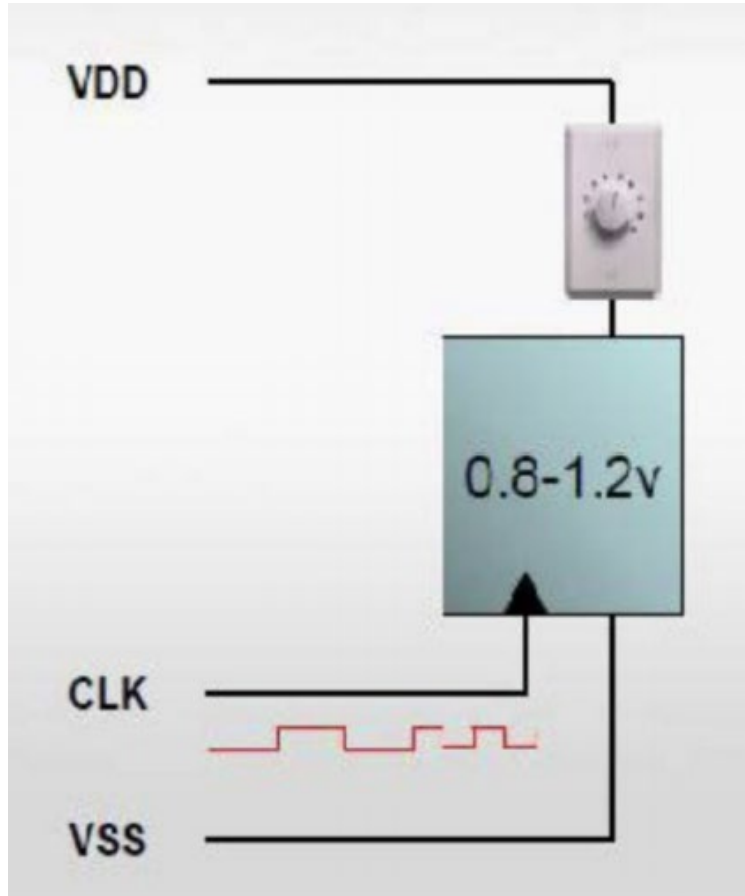
Shutting off voltage to some modules when they are not operating.

MULTI-VOLTAGE DOMAIN :

Some portion of the design which is not doing higher computation can operate at lower voltage.



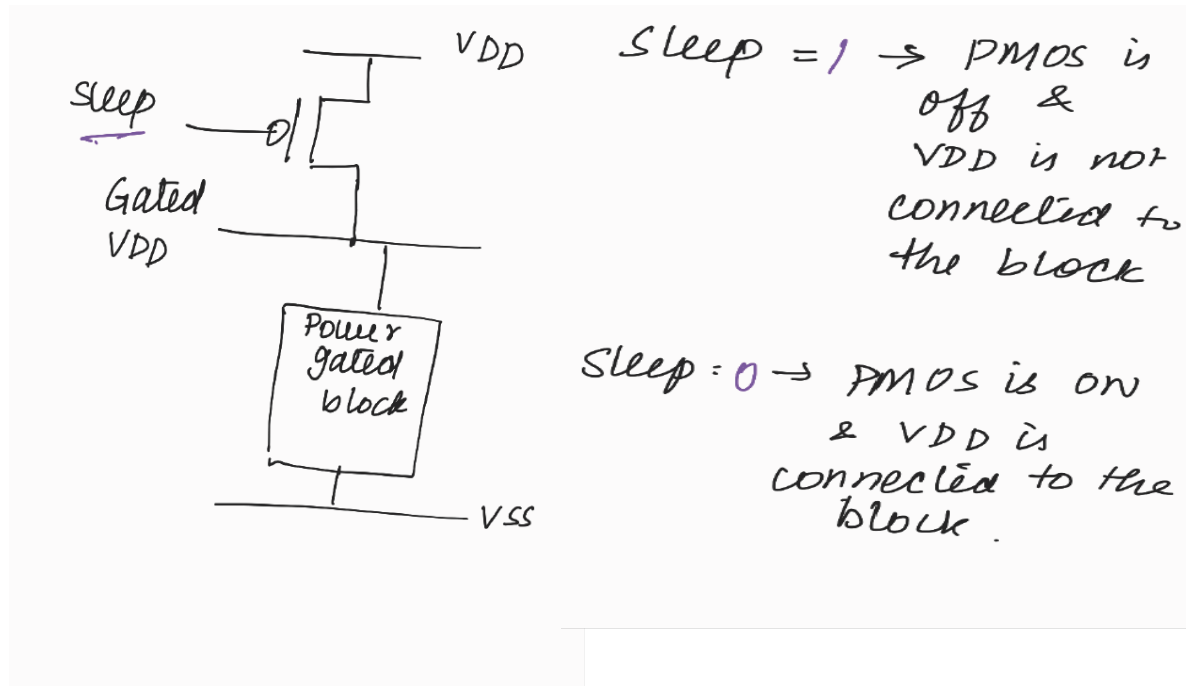
POWER REDUCTION TECHNIQUES (Contd...)



VOLTAGE/FREQUENCY SCALING :

When a module is doing higher computation, we give higher voltage else we give lower voltage.

SPECIAL CELLS REQUIRED FOR LOW POWER DESIGN



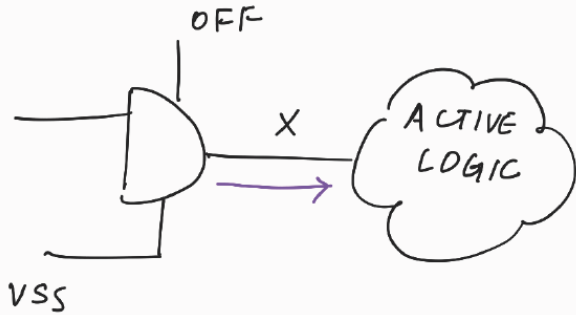
POWER SWITCH

Even when CMOS cells are not switching, they consume leakage power.

Save leakage power by turning off design partitions when that is inactive.

SPECIAL CELLS REQUIRED FOR LOW POWER DESIGN

(Contd...)



ISOLATION CELL :

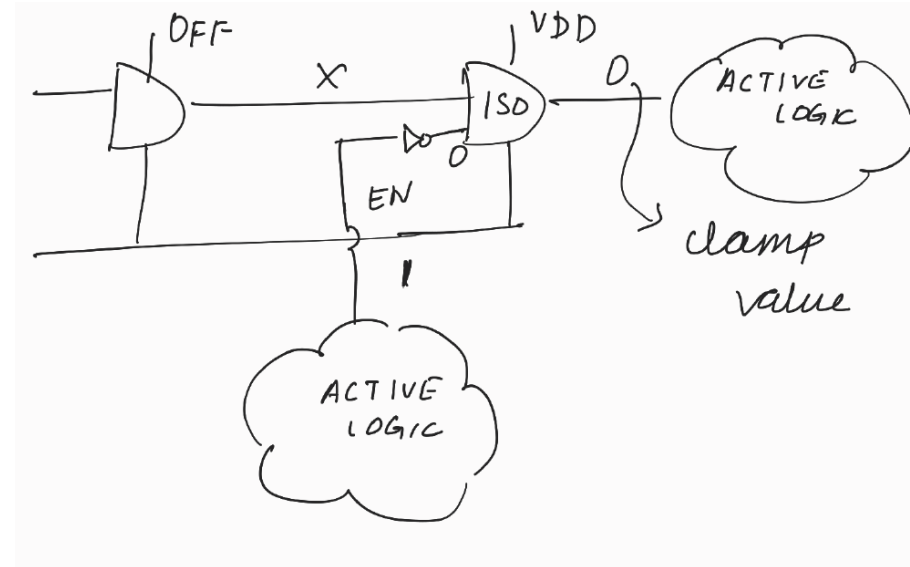
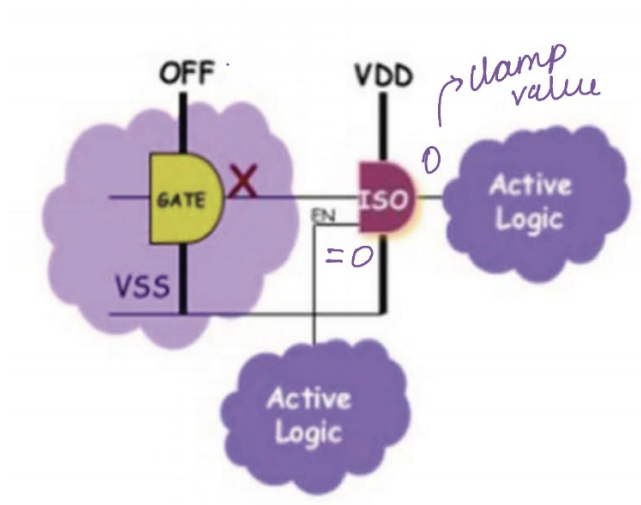
Signal crossing from PD1 to PD2

PD1 => off

PD2 => will not get proper signal.

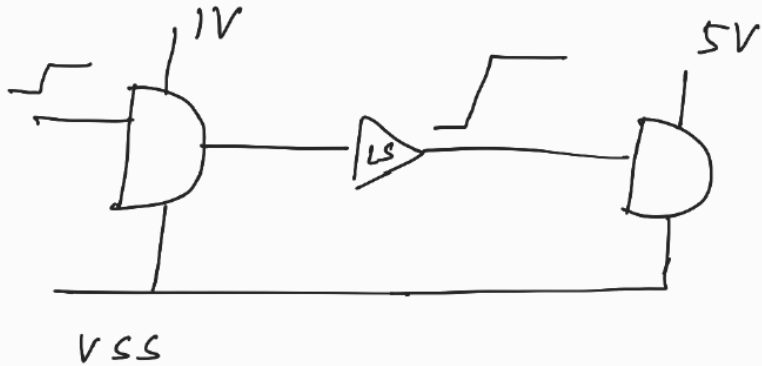
POWER DOMAIN :
It is a logic partition with common power characteristics.

So we add a ISO cell.



SPECIAL CELLS REQUIRED FOR LOW POWER DESIGN

(Contd...)



LEVEL SHIFTERS :

When driving a signal from 1 voltage domain to another voltage domain, logic 0 and 1 should be sensed properly. So we use level shifter.

Assume:

logic 0 of 1V domain : 0 to 0.3V

logic 1 of 1V domain : 0.7V to 1V

logic 1 of 5V domain : above 4V.

∴ whatever logic we are sending from 1V level will be sensed as 0 logic.

So we have to use level shifters.

SPECIAL CELLS REQUIRED FOR LOW POWER DESIGN

(Contd...)

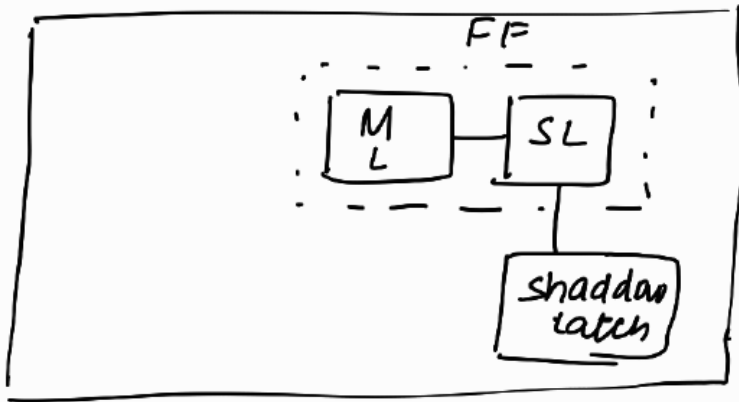
RETENTION REGISTER : It is used during power gating. During power gating, we will not be able to hold old values. Register will go to x (default value)

But during power gating, if some portion of logic is required to hold their old value (which all portions will be decided by the designer), we will use retention register for those portions.

SPECIAL CELLS REQUIRED FOR LOW POWER DESIGN

(Contd...)

RETENTION REGISTER



2 circuits :

- Standard register logic, supplied by primary power (VDD)
- “Shadow latch” retention circuitry, with alternate supply (VDDDB)

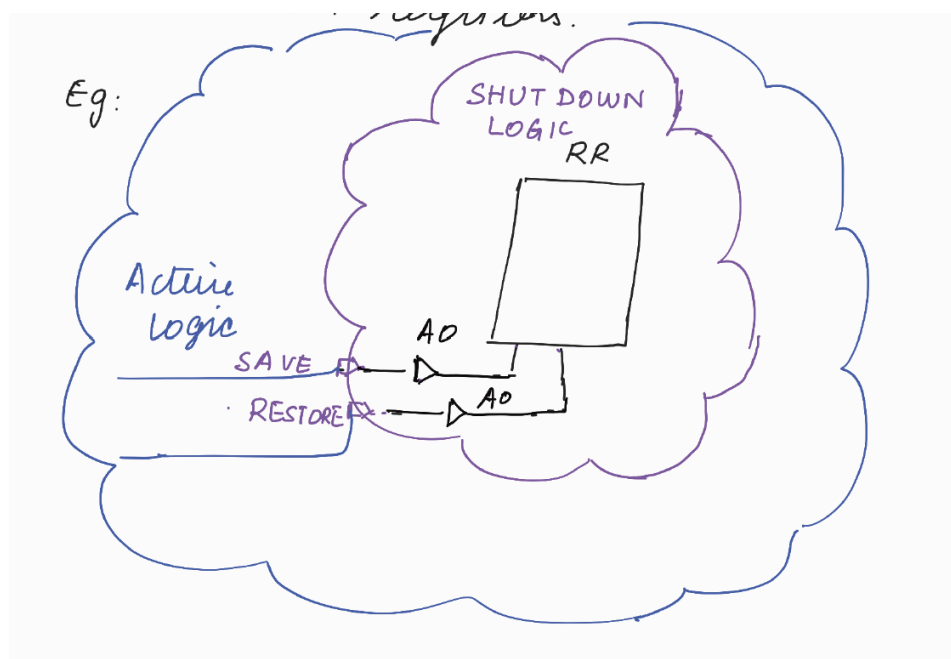
Remains powered to maintain memory of saved state.

Save : transfer FF content into shadow latch during shutdown

Restore : transfer state from shadow latch to FF when powered back on.

SPECIAL CELLS REQUIRED FOR LOW POWER DESIGN (Contd...)

Always-on logic

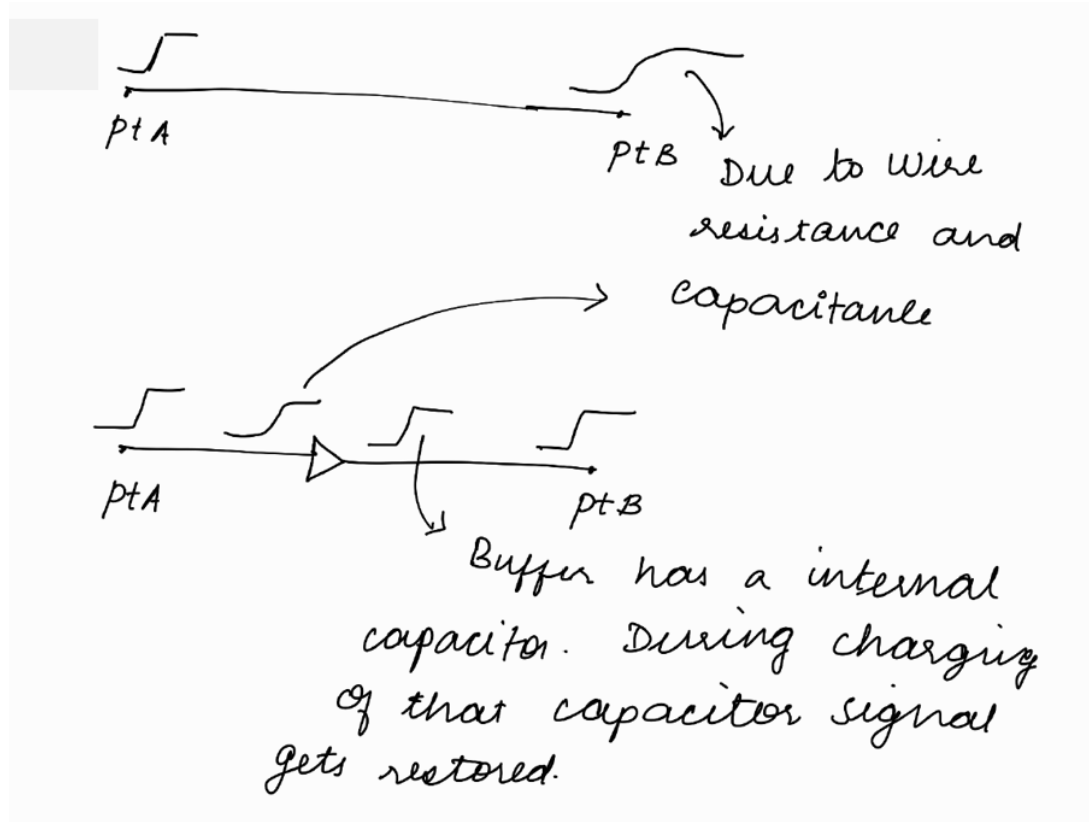


Some logic needs to stay active during lockdown.
Example :

- Path to enable pins of ISO cells
- Save and restore signals of retention registers.

SPECIAL CELLS REQUIRED FOR LOW POWER DESIGN (Contd...)

Normally while forming connection between port and pin of a cell we use buffer to avoid degradation of signals.



But in this case instead of a normal buffer we need to add a AO buffer because it has to be ON when the module is shut down.

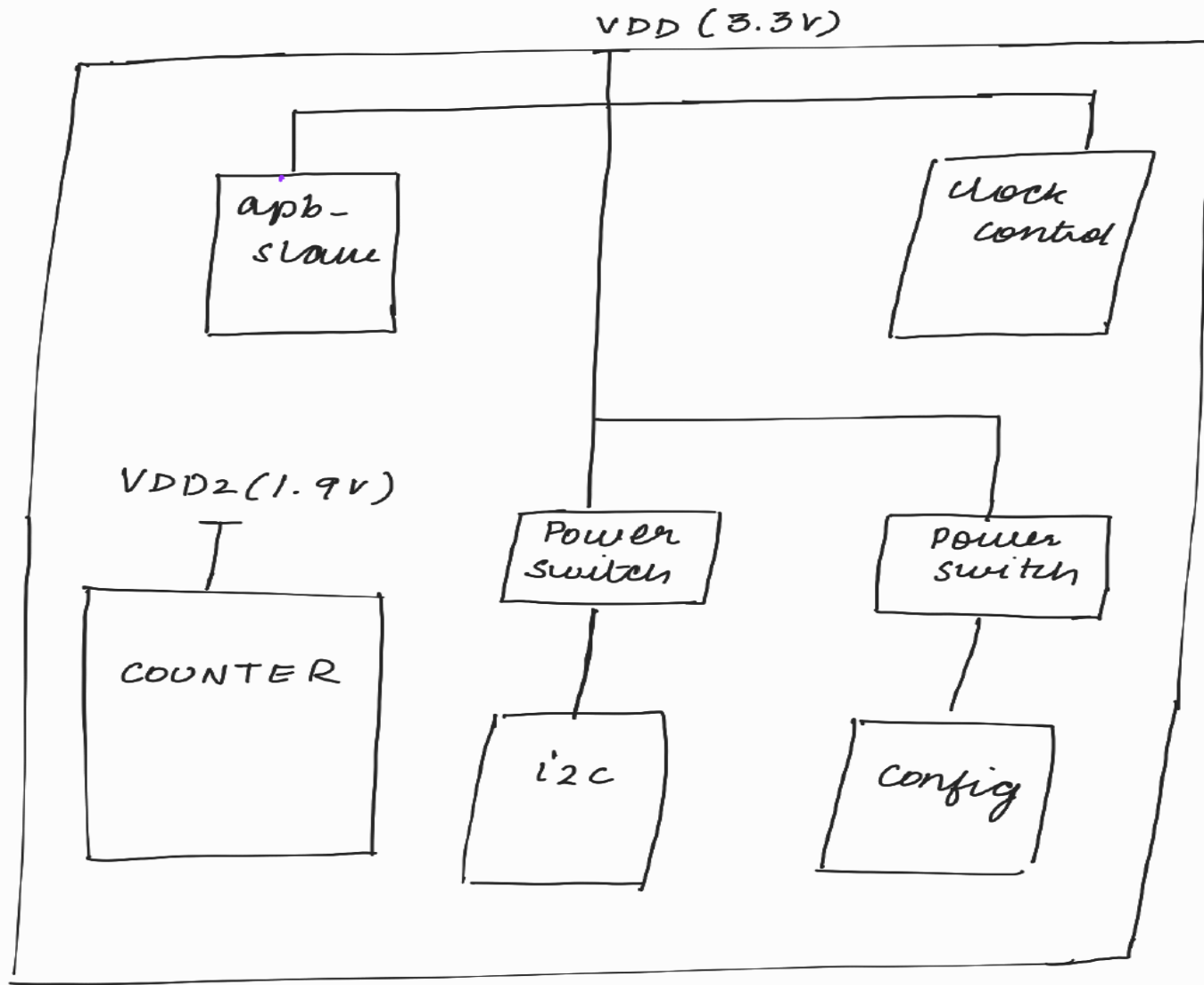
UNIFIED POWER FORMAT (UPF)

- UPF is an IEEE standard for specifying power intent.
- Power intent specifies
 - Power domains
 - Shutdown logic
 - Power State Tables
 - Operating Voltages
 - Usage of special cells (isolation cells, level shifters, power switches, retention registers)

Purpose of UPF :

To implement and to verify low power design techniques we need UPF.

EXAMPLE :



apb-slave } 3.3V, always on
clock control }

i2c, config } 3.3V, but are having power gating (not on always)

counter $\Rightarrow$ 1.9V

These information will be given by chip lead after discussing with architecture team.

POWER DOMAIN :

It is a logic partition with common power characteristics.

- In the above example, we have 4 power domains :
 - APB_slave, clock control
 - Counter
 - I2c
 - Config

Note : i2c and config cannot be grouped because i2c will be on sometimes and config will be on at some other times.

- We need 2 switches.
- We need a level shifter to counter

➤ There are 4 possible Power State Table (PST) scenarios.

PST : It will check for all possible scenarios in our design.

Scenario 1 : all modules on

Scenario 2 : all modules off

Scenario 3 : only i2C is off

Scenario 4 : only config is off

NOTE :

Our example, (asynchronous FIFO), is a single module, a single power domain is used for asynchronous FIFO.

THANK YOU